

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1405

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

*I **Sudharsan D/192524063** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sudharsan D/192524063

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Q1. E-commerce Order Processing System.

Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

An e-commerce platform processes customer orders. Variables include product price, quantity, tax, and total amount.

Task

1. Write declarations for all variables.
2. Convert the following high-level code into three-address intermediate code:
 total = price * quantity
 tax_amount = total * tax
 final_amount = total + tax_amount
3. Optimize the intermediate code (if possible).

Q2. Banking Loan Approval System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A bank evaluates loan eligibility based on:

- Salary > 30,000
- Credit Score > 700

Task

1. Convert the condition into Boolean expression intermediate code.
2. Generate flow of control using backpatching technique.
3. Represent using labelled instructions.

Q3. Traffic Signal Control System

Concepts Covered: Case Statements

Problem Statement

A smart traffic system changes signals based on input:

```
switch(signal)
case 1: RED
```

case 2: YELLOW

case 3: GREEN

Task

1. Convert the case statement into intermediate code.
2. Use jump tables or conditional jumps.
3. Explain efficiency of your approach.

Q4. Online Payment Gateway

Concepts Covered: Procedure Calls

Problem Statement

A payment system calls functions:

```
validateCard()  
processPayment()  
generateReceipt()
```

Task

1. Generate intermediate code for procedure calls.
2. Show:
 - Parameter passing
 - Call and return sequence
3. Use stack-based representation.

Q5. Smart Home Automation System

Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

A smart home system turns ON AC if:

- Temperature > 30 AND Room Occupied

Task

1. Write Boolean expressions.
2. Convert into intermediate code with short-circuit evaluation.
3. Apply backpatching to handle jumps.
4. Assign final output to variable AC_status.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q2. Banking Loan Approval System:

The bank approves a loan when:

Salary > 30000 AND CreditScore > 700

The question specifically requires Boolean intermediate code, backpatching and labelled instructions.

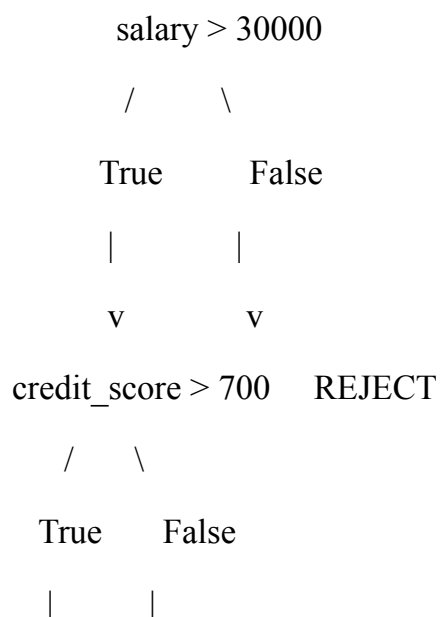
1. Boolean Expression:

$B = (\text{salary} > 30000) \text{ AND } (\text{credit_score} > 700)$

2. Three-Address Boolean Code:

1. if salary > 30000 goto 3
2. goto 7
3. if credit_score > 700 goto 5
4. goto 7
5. loan_approved = 1
6. goto 8
7. loan_approved = 0
8. end

Flow:



v v
APPROVE REJECT

3. Backpatching:

Initially:

1. if salary > 30000 goto _
2. goto _
3. if credit_score > 700 goto _
4. goto _

After backpatching:

1. if salary > 30000 goto 3
2. goto 7
3. if credit_score > 700 goto 5
4. goto 7
5. loan_approved = 1
6. goto 8
7. loan_approved = 0
8. end

Engineering interpretation:

The use of **short-circuit evaluation** avoids checking the credit score when the salary condition already fails. This reduces unnecessary computation and makes the loan eligibility decision efficient.

Q4. Online Payment Gateway

The payment system contains:

```
validateCard()  
processPayment()  
generateReceipt()
```

The question requires procedure-call intermediate code, parameter passing, call/return sequence and stack representation.

1. Intermediate Code:

Assume the payment system receives:

card
amount

Three-address procedure-call code:

1. param card
2. t1 = call validateCard, 1
3. if t1 == 0 goto 12
4. param amount
5. t2 = call processPayment, 1
6. if t2 == 0 goto 12
7. param t2
8. t3 = call generateReceipt, 1
9. result = t3
10. goto 13
12. result = "Payment Failed"
13. END

2. Parameter Passing:

For `validateCard()`:

param card
call validateCard

For `processPayment()`:

param amount
call processPayment

For `generateReceipt()`:

param t2
call generateReceipt

Call and Return Sequence:

Caller

|

| param card

| call validateCard

v

validateCard()

|

| return validation_result

v

Caller

|

| param amount

| call processPayment

v

processPayment()

|

| return payment_result

v

Caller

|

| param payment_result

| call generateReceipt

v

generateReceipt()

|

| return receipt

v

Caller

3. Stack-Based Representation:

Before calling `validateCard()`:

```
+-----+
| Return Address |
+-----+
| card          |
+-----+
| Parameters     |
+-----+
```

During `processPayment()`:

```
+-----+
| Return Address |
+-----+
```



```

| amount      |
+-----+
| Previous Frame |
+-----+

```

During `generateReceipt()`:

```

+-----+
| Return Address |
+-----+
| payment_result |
+-----+
| Previous Frame |
+-----+

```

Engineering interpretation:

The stack provides temporary storage for parameters, return addresses and local execution information. Each procedure receives its parameters and returns its result to the calling procedure.

Q5. Smart Home Automation System

The condition is:

Temperature > 30 AND Room Occupied

The AC should be turned ON only when **both conditions are true**.

1. Boolean Expression:

$B = (\text{temperature} > 30) \text{ AND } (\text{room_occupied} == \text{true})$

2. Three-Address Code with Short-Circuit Evaluation:

1. if temperature > 30 goto 3
2. goto 7
3. if room_occupied == true goto 5
4. goto 7
5. AC_status = ON
6. goto 8
7. AC_status = OFF
8. END

Why short-circuit evaluation?

Because the expression uses:

AND

If:

temperature <= 30

then the complete expression is automatically false.

Therefore, there is no need to check:

room_occupied

This saves computation.

3. Backpatching:

Initially:

1. if temperature > 30 goto _
2. goto _
3. if room_occupied == true goto _
4. goto _

Backpatch the true and false destinations:

1. if temperature > 30 goto 3
2. goto 7
3. if room_occupied == true goto 5
4. goto 7
5. AC_status = ON
6. goto 8
7. AC_status = OFF
8. END

Backpatching Table:

Instruction	Condition	Destination
1	temperature > 30	3
2	False	7
3	room_occupied == true	5
4	False	7

5	AC ON	—
7	AC OFF	—

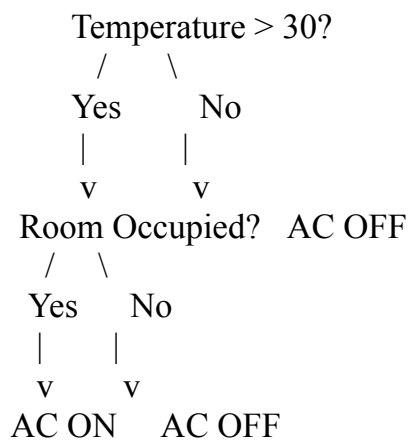
4. Final Output:

```

if temperature > 30 AND room_occupied == true
    AC_status = ON
else
    AC_status = OFF

```

Flow:



Engineering interpretation:

The system uses **short-circuit Boolean evaluation and backpatching** to efficiently control the air conditioner. The AC is switched ON only when the temperature exceeds 30°C and the room is occupied.